

SAGA: Synthetic Audit Log Generation for APT Campaigns

Yi-Ting Huang¹, Ying-Ren Guo², Yu-Sheng Yang³, Guo-Wei Wong⁴, Yu-Zih Jheng⁵, Yeali S. Sun⁶,
Jessemyn Modini, Timothy Lynar⁷, and Meng Chang Chen⁸

Abstract—With the increasing sophistication of Advanced Persistent Threats (APTs), the demand for effective detection and mitigation strategies and methods has escalated. Program execution leaves traces in the system audit log, which can be analyzed to detect malicious activities. However, collecting and analyzing large volumes of audit logs over extended periods is challenging, further compounded by insufficient labeling that hinders their usability. Addressing these challenges, this paper introduces SAGA (Synthetic Audit log Generation for APT campaigns), a novel approach for generating fine-grained labeled synthetic audit logs that mimic real-world system logs while embedding stealthy APT attacks. SAGA generates configurable audit logs for arbitrary duration, blending benign logs from normal operations with malicious logs based on the definitions the MITRE ATT&CK framework. Malicious audit logs follow an APT lifecycle, incorporating various attack techniques at each stage. These synthetic logs can serve as benchmark datasets for training machine learning models and assessing diverse APT detection methods. To demonstrate the usefulness of synthetic audit logs, we ran established baselines of event-based technique hunting and APT campaign detection using various synthetic audit logs. In addition, we show that a deep learning model trained on synthetic audit logs can detect previously unseen techniques within audit logs.

Index Terms—Technique hunting, APT campaign, provenance-based detection, synthetic audit log, deep learning.

I. INTRODUCTION

THE emergence of Advanced Persistent Threats (APTs), such as Fancy Bear (APT28) and Lazarus Group Elfin (APT33), poses significant challenges to the cybersecurity community. APT campaigns, typically orchestrated by advanced threat actors with economic or political motives, involve multiple stages, including gaining initial access to a victim environment, remaining undetected for extended periods, exfiltrating sensitive data, and compromising system integrity. Despite progress in

malware analysis [1], [2], effective detection and mitigation of APT attacks remains a critical priority. A key obstacle in APT threat studies is the lack of a widely recognized benchmark dataset [3], which is essential to develop AI/ML-based solutions and objectively evaluate methods. This challenge motivates the study of synthetic audit log generation in this work.

System audit logs, which record traces of program execution, play a crucial role in log analysis for threat detection. For example, MITRE ATT&CK Evaluations [4] provide emulation plans that allow cybersecurity providers to assess their product's ability to detect simulated cyber attacks through system logs. With the advancement of data-driven artificial intelligence methods, it has become increasingly effective in addressing certain cyber threats, such as attack analysis [5], [6], [7], which offer the potential for timely Technique detection. These approaches typically rely on high-quality labeled audit logs for model training. In addition, it allows for the enhancement and comparison of various methods.

However, labeling large volumes of audit logs over long periods poses challenges, especially when APT attacks lack obvious indicators. Researchers often rely on publicly available datasets such as the DARPA Transparent Computing Engagement [8], Operationally Transparent Cyber (OpTC) [9], and LANL's dataset [10], as well as smaller datasets that are rarely shared due to privacy and proprietary concerns. Moreover, some datasets provide fine-grained labels on malicious entities [5] or processes [11], while others [8], [9] only report attack scenarios over time intervals, without linking them to specific system events. This shortage of labeled logs limits a deeper understanding of attack behaviors and restricts the development of AI/ML-based cybersecurity defense studies [12], [13].

The absence of such audit logs motivates the development of methods for generating configurable synthetic audit logs that fulfill the following key requirements.

- **Realism:** Synthetic audit log should closely resemble the real audit logs.
- **Scenario coverage:** They should represent a broad spectrum of APT campaigns, including recent and emerging threats.
- **Detailed labeling:** Logs should include fine-grained labels such as specific Techniques or equivalents mapped to events in the audit log.
- **Higher-Level representation:** Along with raw events, they should offer higher-level abstractions, facilitating easier understanding, examination, and communication.

Received 20 November 2024; revised 30 October 2025; accepted 28 November 2025. Date of publication 5 December 2025; date of current version 12 March 2026. This work was supported in part by CITI, Academia Sinica and in part by MOST under Grant NSTC 114-2221-E-011-080-MY3, Grant 114-2634-F-001-001-MBK, and Grant 114-2221-E-001-018. (Corresponding author: Yi-Ting Huang.)

Yi-Ting Huang is with the National Taiwan University of Technology and Science, Taipei 106, Taiwan (e-mail: ythuang@mail.ntust.edu.tw).

Ying-Ren Guo and Meng Chang Chen are with Academia Sinica, Taipei City 115201, Taiwan (e-mail: mcc@iis.sinica.edu.tw).

Yu-Sheng Yang, Guo-Wei Wong, Yu-Zih Jheng, and Yeali S. Sun are with National Taiwan University, Taipei City 106319, Taiwan.

Jessemyn Modini and Timothy Lynar are with the University of New South Wales, Kensington 2052, Australia.

This article has supplementary downloadable material available at <https://doi.org/10.1109/TDSC.2025.3640696>, provided by the authors.

Digital Object Identifier 10.1109/TDSC.2025.3640696

- **Flexibility:** The framework should be able to generate logs of any duration and include any number of APT campaigns.
- **Diversity:** The framework should be able to generate previously unseen logs, which would benefit model training and examination.

This study introduces SAGA (Synthetic Audit Log Generation for APT Campaigns), a framework for generating configurable synthetic audit logs. SAGA is aligned with the MITRE ATT&CK framework and terminologies, chosen for its openness, comprehensiveness, and availability. The generated synthetic audit log contains both benign and malicious (APT campaigns) audit logs, with benign logs representing normal system and user operations and malicious logs constructed from predefined Technique templates. Each APT campaign in the logs, comprising stages/tactics with corresponding techniques, which are reflected in the audit logs.

The specific tasks of this study include:

- **Obtaining attack scenarios:** Use a red team emulator to collect and label logs with technique identifiers based on the MITRE ATT&CK framework.
- **Designing the SAGA framework:** Develop the SAGA framework and workflow to address the aforementioned challenges.
- **Synthesizing configurable audit logs:** Create synthetic audit logs that combine APT attack scenarios and benign behaviors, resembling real data logs.
- **Verifying SAGA usefulness:** Run existing systems using the SAGA synthetic audit log as input to demonstrate its effectiveness. Furthermore, show that models trained on the synthetic audit log can successfully identify previously unseen malicious behaviors in new datasets.

To demonstrate the usefulness of synthetic audit logs, we tested established baselines and the existing methods across various applications, including APT intrusion detection, technique hunting, and APT campaign detection. The objective of this empirical study is to demonstrate that SAGA synthetic audit logs can be used effectively and meaningfully in cybersecurity-related audit log applications. The focus is not on comparing the performance or goodness of these methods, as they target different objectives and operate in distinct contexts, making direct comparison uninformative.

II. BACKGROUND AND MOTIVATION

In this section, we introduce the audit log and its related issues using a running example of an APT campaign.

A. System Audit Log

System audit logs are records of system events such as file access, process creation and termination, and network access. Each system event depicts the interaction between two system entities, typically represented as $\langle \text{subject}, \text{operation}, \text{object} \rangle$, where the subject refers to a process entity or a program, and the object represents a system entity such as a process, file, registry, or network socket. The operation signifies the action performed by the subject on the object. For example, the event $\langle \text{WinRAR.exe}, \text{CreateFile}, \text{IOC_09_11.rar} \rangle$ indicates that the

Time of Day	PID	Process Name	Operation	Path
56:50.9	1756	firefox.exe	QueryOpen	C:\Users\apple\Downloads\IOC_09_11.rar
56:50.9	1756	firefox.exe	CreateFile	C:\Users\apple\Downloads\IOC_09_11.rar
56:50.9	1756	firefox.exe	CloseFile	C:\Users\apple\Downloads\IOC_09_11.rar
56:55.5	3576	Explorer.EXE	Process Create	C:\Program Files\WinRAR\WinRAR.exe
56:55.5	9912	WinRAR.exe	Process Start	
56:55.5	9912	WinRAR.exe	Thread Create	
56:55.5	9912	WinRAR.exe	Load Image	C:\Program Files\WinRAR\WinRAR.exe
56:55.5	9912	WinRAR.exe	Load Image	C:\Windows\System32\ntdll.dll
56:55.6	9912	WinRAR.exe	QueryOpen	C:\Users\apple\Downloads\IOC_09_11.rar
56:55.6	9912	WinRAR.exe	CreateFile	C:\Users\apple\Downloads\IOC_09_11.rar
56:55.6	9912	WinRAR.exe	CloseFile	C:\Users\apple\Downloads\IOC_09_11.rar

Fig. 1. Example of audit log events captured by *Procmon*.

WinRAR process creates a file named *IOC_09_11.rar*. System events, along with their subjects and objects, include attributes such as meta-information, properties, and other details provided by the system, which can further describe the event and context. These details will be discussed in the subsequent section. System audit logs can be obtained using built-in kernel event logging mechanisms such as *Windows Event Tracing (ETW)* and *Linux Audit*, or through additional tools like *Procmon*, *Sysmon*, and *CamFlow*.

In this study, we use *Procmon* as the primary logging tool. *Procmon*, short for Process Monitor is a Windows utility that monitors and records real-time system activities, including process actions, file system activities, registry changes, and more. Fig. 1 provides an example of the captured system events. The first row records the event $\langle \text{firefox.exe}, \text{QueryOpen}, \dots, \text{IOC_09_11.rar} \rangle$, along with attributes such as timestamp, PID, and object value, which indicate a query for the existence of the RAR file. The audit log is proved useful for analyzing malicious attempts and APT attacks, offering insights into their characteristics.

B. APT Attack Lifecycle

Frameworks such as Lockheed Martin's cyber kill chain [14], MITRE ATT&CK framework [15], and Mandiant adversary lifecycle [16] define the lifecycle of cyber attacks, helping defenders understand the collective operations and intention of APT attacks. This study uses the Mandiant adversary lifecycle, depicted in Fig. 2(a), as a reference for the APT attack lifecycle, although other frameworks can be equally applicable. Each stage/tactic encompasses associated Techniques, and each Technique may have several implementations/abilities. In addition, this lifecycle framework helps to generate and analyze synthetic audit logs. Note that in this paper, we consider Technique and Sub-Technique equally and use the term Technique to represent both.

A typical APT attack consists of multiple stages, including initial compromise, establishing foothold, internal reconnaissance, escalating privileges, moving laterally, maintaining presence, and ultimately complete mission. Note that the stages of internal reconnaissance, escalating privileges, moving laterally, and maintaining presence may occur multiple times, and not all four stages are necessary in every instance. Fig. 2(a) shows the lifecycle stages corresponding to the APT28 campaign.

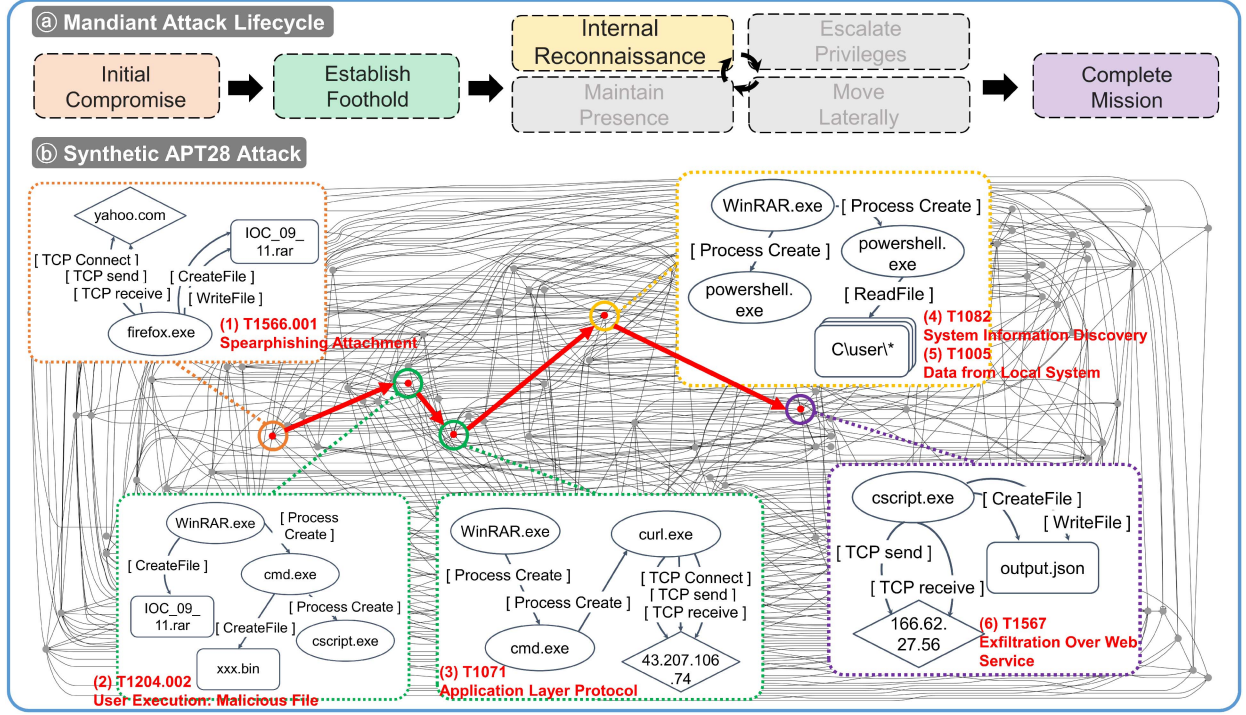


Fig. 2. (a) Mandiant attack lifecycle [16] of APT28. (b) The APT28 attack provenance graph, derived from [17], presents five malicious events labeled with six techniques, which are mapped to the corresponding stages of the attack lifecycle. In each zoomed-in box, the dotted boundary color corresponds to its specific stage in the attack lifecycle. Rectangular nodes represent files, diamond-shaped nodes represent sockets, oval nodes denote processes, and the edges illustrate the causal relationships between the entities. The red arrow edges indicate the sequence of malicious events.

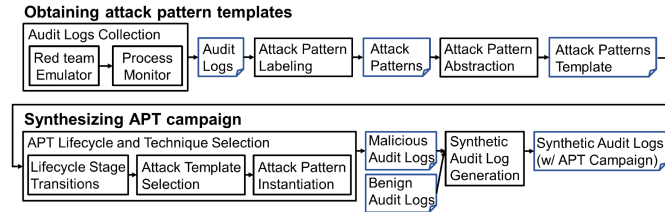


Fig. 3. SAGA workflow.

C. Running Example

We refer to APT28 threat intelligence reports [17] to construct the attack scenario illustrated in Fig. 2, which serves as the running example in this work. Fig. 2(a) outlines its attack lifecycle, while Fig. 2(b) shows the corresponding APT audit log. In the initial stages of the attack, phishing emails containing malicious attachments are sent to the victim to download the attachment *IOC_09_11.rar*. This activity is designated as the Technique *T1566.001 Spearphishing Attachment*, as (1) in Fig. 2(b), of the attack lifecycle stage *Initial Compromise*. This malicious attachment exploits a vulnerability in WinRAR versions below 6.23 (as mentioned in CVE-2023-38831). When the victim clicks on the *IOC_09_11.rar* file, it triggers subsequent malicious activities designated as *T1204.002 User Execution: Malicious File* of the attack lifecycle *Establish Foothold*, as (2) in Fig. 2(b).

The attack proceeds by establishing a reverse shell between the attacker and the victim, designated as *T1071 Application Layer Protocol* of the *Establish Foothold* stage, marked as (3) in the figure. Next, it steals information such as local state files, corresponding to *T1082 System Information Discovery* and *T1005 Data from Local System*, marked as (4) and (5) of the *Internal Reconnaissance* stage. Finally, the stolen information is uploaded to a web *servicewebhook.site*, designated as *T1567 Exfiltration Over Web Service* of the *Complete Mission* stage, shown as (6) in Fig. 2(b).

III. SAGA GENERATIVE MODEL

The SAGA generative model involves collecting audit logs from a red-team emulator to create a repository of attack pattern templates with fine-grained labels. These templates are then used, following lifecycle selection instructions, to generate synthetic audit logs that embed APT campaigns. Fig. 3 provides an overview and the workflow of SAGA, which will be discussed in detail in the following subsections. To mitigate the risk of detection models overfitting on artificial patterns, SAGA introduces variability in both attack implementation in Section III-A and artifact representation in Section III-C3. In Section III-A, SAGA collects attack pattern from a red-team emulation platform that supports multiple abilities per Technique, enabling the generation of diverse execution paths and behavioral variants. In Section III-C3, SAGA incorporates entity-level variability

by replacing generalized entities with hyponyms derived from real-world malware samples and synthetic variants. This combination of implementation diversity and entity-level variation helps generate a wide range of realistic log instances.

A. Attack Pattern Collection and Labeling

The first task of SAGA is to collect labeled audit logs, as depicted in Fig. 3. The Red Team Platform, such as CALDERA [18], [19] and Atomic Red Team [20], simulates real-world adversaries in a controlled environment. In this study, we use CALDERA to emulate real-world attacks by adversaries. Our choice is based on its open source nature and alignment with the MITRE ATT&CK framework. To deploy CALDERA, we configure a master server and set up Remote Access Tool (RAT) clients on a compromised host. An *ability* of CALDERA refers to a specific implementation of an ATT&CK Technique, representing a discrete adversarial behavior. After acquiring audit logs from a red team emulator and process monitor, the next step is to extract the relevant attack patterns and assign appropriate Technique labels. CALDERA-generated reports provide details for each executed attack, including process IDs, commands used, and associated technique IDs. We employ a semi-automated labeling method to annotate audit logs based on handcrafted rules tailored to specific attack techniques.

To develop specific rules for each ability, three authors applied their expert knowledge based on the MITRE ATT&CK framework [15] to define rules for identifying attack-related events. In the case of T1566.001, the ability of T1566.001 involves using the *Invoke-WebRequest* cmdlet of PowerShell, which indicates network activity connecting to the GitHub website. Furthermore, the flag *-OutFile* and parameter *\\$env:TEMP\PhishingAttachment.xlsm* suggests that a file named *PhishingAttachment.xlsm* will be created on the target system. To ensure accurate labeling, rules are designed and applied using both forward and backward tracing, since not all events generated during an attack should be considered malicious and labeled with the Technique identifier. Some may represent background or system activities. Only events exhibiting these specific malicious actions and artifacts are considered malicious for this particular ability, and custom labeling rules are crafted to capture this behavior. We validated these rules using a set of manually labeled logs and through cross-verification via code review among the three authors.

When given a CALDERA-generated report, SAGA uses the attack-related Process ID (PID) as the root to construct a comprehensive process family tree that captures all associated processes. By mapping these process IDs to the corresponding audit logs and applying handcrafted rules, SAGA can accurately assign the appropriate technique ID to the relevant log entries.

B. Attack Pattern Templates Abstraction

Each attack pattern corresponds to a specific instance of a Technique with particular parameter values. To enable the generation of a wider variety of attack patterns, it is essential to abstract these patterns into a more abstract form. These abstractions can then be instantiated with legitimate parameter values, allowing

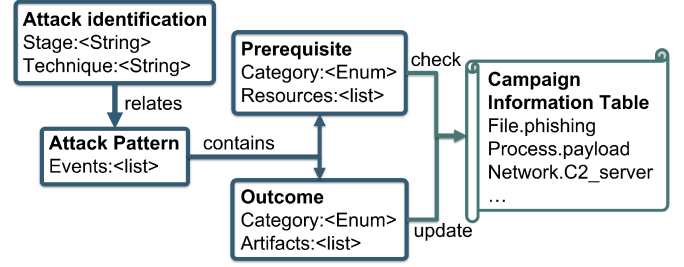


Fig. 4. Attack pattern template model.

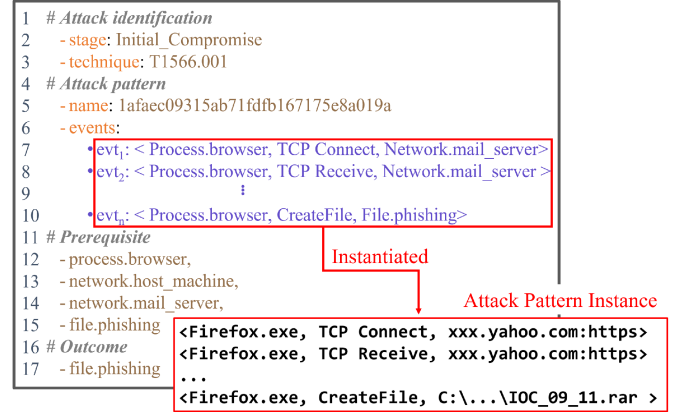


Fig. 5. APT28 Attack pattern template example.

for the creation of new attack patterns. This approach facilitates greater flexibility and diversity in the generation of new attack instances.

We abstract each attack pattern into an attack template, which specifies its prerequisites, outcomes, timings, and the corresponding stage and Technique in the attack lifecycle. This meta-information serves as a blueprint for generating diverse attack instances while maintaining consistency among the generated instances. Fig. 4 shows the attack template model, which is explained in more detail with an example in Fig. 5. Algorithm 1 shows the details of template generation.

Attack Identification: The entity *Attack Identification* includes the attack *stage* and *Technique*, which together define the identity of the attack pattern template, as shown in Fig. 4. For example, lines 1–3 of Fig. 5 illustrate an attack pattern template associated with Technique *T1566.001 Spearphishing Attachment* and the stage *Initial Compromise* in the adversary lifecycle.

Attack Pattern: The entity *Attack Pattern* is associated with an attack identity and consists of a sequence of events generated by a specific ability, categories and descriptors of artifacts presented in the event. In this context, the category provides a broad classification, while the descriptor offers a more specific identification. For example, if an event contains the artifact *Firefox*, it falls into the category *process* with the descriptor *browser*. To support the generalization of attack pattern templates, Table I outlines a collection of categories and descriptors used in SAGA, which can be considered as forming a hypernym relationship. The table outlines six categories, File, Network, Cmdline, Process,

TABLE I
CATEGORIES AND DESCRIPTORS USED IN THIS STUDY

Category	Descriptor	Category	Descriptor
File	ADS(Alter. data streams)	Process	Browser*
	Exfiltration Folder		Explorer*
	Payload		Payload
	Payload Copy	Registry	Phishing
	Phishing		Key
	Recon		Subkey
	Script	System	Firewall Port*
	Shortcut		Firewall Rule
Network	C2		Host*
	Host IP*		Password*
	Host Machine*		Product
	Mail Server		Proxy Port*
	Payload URL		Service
	Script URL		Task
Cmdline	Command		Time*
	Content		User*
	Message		Userdomain

* indicates the instances of descriptor are generated by using Faker [21].

Algorithm 1: Generate Attack Pattern Template.

Input: Attack pattern A and events E

Output: Attack pattern template P

- 1: Let TAB that stores known system entities and their descriptors and categories, as defined in Table I.
 - 2: Initialize P with the stage and Technique, as described in Subsection “Attack Identification”.
 - 3: **for** each event e in E **do**
 - 4: **for** each system entity s in E **do**
 - 5: **if** s already used **then**
 - 6: Replace s with its descriptor d from TAB .
 - 7: **else**
 - 8: Find its descriptor d and replace s .
 - 9: Set the descriptor as a prerequisite.
 - 10: **end if**
 - 11: Set d as an outcome if it still exists.
 - 12: Insert e into P .
 - 13: **end for**
 - 14: **end for**
 - 15: **return** P
-

Registry, and System, each subdivided into specific descriptors. Refer to lines 3-6 of Algorithm 1.

Subsequently, additional attack instances can be generated by instantiating these attack pattern templates during the generation of synthetic audit logs. Using the same running example, three system entities (artifacts) are involved in phishing: *Firefox* accessing *Microsoft Exchange* and downloading *IOC_09_11.rar* through the browser. These entities can be substituted with hypernyms (as shown in line 12 of Algorithm 1 and lines 7-10 of Fig. 5), such as replacing *Firefox* with *Process.browser* and *IOC_09_11.rar* with *File.phishing*.

Prerequisite: The entity *Prerequisite* specifies conditions related to the system resources required for the execution of an attack pattern (lines 8-9 of Algorithm 1), such as *process.browser* and *file.phishing* (as shown in lines 12-15 of Fig. 5) of the attack pattern template T1566.001 of APT28. To generate a specific malicious audit log (i.e., an attack pattern), a campaign

information table records previously executed attacks and the system entities utilized, as well as the system entities provided by the external environment, to facilitate *Prerequisite* matching. An attack pattern is generated only when all *Prerequisite* are satisfied; more details are given in Section III-C2.

Outcome: The entity *Outcome* lists the artifacts produced after executing this attack (line 11 of Algorithm 1), such as a phishing file (as indicated on line 17 of Fig. 5), collected sensitive information, and changes in system state. In a complete APT attack chain, *Outcome* may serve as prerequisites for subsequent attack patterns. Therefore, defining the *Outcome* within the template is crucial, as it not only impacts the immediate effects of the attack, but also has long-term implications for future attack strategies and defensive measures.

C. APT Lifecycle and Technique Selection

SAGA supports both random and user-specified APT campaign generation, and in this subsection, the random APT campaign generation is introduced. The generation of an APT campaign is conditioned on the adversary lifecycle, involving lifecycle stage transitions, attack template selections, and attack pattern instantiation. The transition process of the lifecycle stage determines the subsequent stage according to the current status. An example of an adversary Mandiant lifecycle is shown in Fig. 2(a). The attack template selection process chooses the attack templates for each stage, while the artifact substitution process instantiates the events for an attack pattern.

1) *Lifecycle Stage Transitions:* As described in Section II-B, threat actors follow a systematic, strategic approach, executing actions sequentially. An APT campaign begins with two initial stages: *initial compromise* (IC) and *establish foothold* (EF). This is followed by an incubation period consisting of four stages: *escalate privileges* (EP), *internal reconnaissance* (IR), *move laterally* (ML), and *maintain presence* (MP). The incubation period may occur once or multiple times, and within each incubation period, individual stages may be omitted or repeated. Finally, the last stage, *complete mission* (CM), is executed once or not at all, marking the successful completion of the attack. The Context-Free Grammar representation of a campaign generation is presented in Table II. The lifecycle is defined in line 6. As an example, in Fig. 2(b), the stages of APT28 include Initial Compromise, Establish Foothold, Internal Reconnaissance, and Complete Mission.

2) *Attack Template Selection:* For each stage of an APT campaign, the number of attacks can be either randomly determined or specified by the user. An attack template (TQ) may also be randomly selected based on the current stage and prerequisites, or as specified by the user. Note that there may be benign events (BE) preceding the attack pattern templates (TQ) that the operation is described in line 8 of Table II. In line 8, malicious audit logs (MA) consist of either a sequence of benign events (BE), the selected attack pattern instance ($In(TQ)$), malicious audit logs (MA), or sequences of benign events (BE) and the selected attack pattern instance ($In(TQ)$). Next, the abstracted arguments of the template are replaced with artifacts to generate diverse audit events, as the instantiation of an attack pattern

TABLE II
CONTEXT FREE GRAMMAR FOR CAMPAIGN GENERATION

1.	$SA \text{ :- } C_m(SA, MA) \mid C_b(SA, BA) \mid MA \mid BA \mid \emptyset$
2.	$BA \text{ :- } EVT$
3.	$EVT \text{ :- } evt + EVT \mid evt \mid \emptyset$
4.	
5.	$/* O+ : \text{zero time or more, } 1+ : \text{one time or more, } 0,1 : \text{zero or one time } */$
6.	$LC \text{ :- } IC + EF + (EP^{0+} + IR^{0+} + ML^{0+} + MP^{0+})^{1+} + CM^{0,1}$
7.	
8.	$MA \text{ :- } BE + In(TQ) + MA \mid BE + In(TQ)$
9.	$BE \text{ :- } EVT$
10.	$In(TQ) \text{ :- } In(TQ) \text{ if } TQ.prerequisites \text{ are satisfied} \mid \emptyset$

Notation: SA = synthetic audit log, MA = malicious audit log, BA = benign audit log, LC = APT lifecycle, IC = initial compromise, EF = establish foothold, EP = escalate privileges, IR = internal reconnaissance, ML = move laterally, MP = maintain presence, CM = complete mission, BE = a sequence of benign events, EVT = event sequence, TQ = attack template, $In(TQ)$ = attack pattern instance

described in line 10 ($In(TQ)$) and elaborated in Section III-C3. The construction of the malicious audit log continues until all stages have been completed.

A campaign information table, as depicted in Fig. 4, is created to maintain information about the executed attack templates, including their associated lifecycle stage, as well as the system entities used and their statuses. Given the current stage, the prerequisites of a template are examined against the campaign information table to determine whether they are satisfied. For example, if a threat actor uses a phishing email template during the initial compromise stage, the campaign information table will record the details of the phishing email template, including the entity *File.phishing*. In the second stage (EF), the templates for this stage will refer to the campaign information table to check whether their prerequisites are met. Only if the prerequisites are met can they be considered potential templates for this APT campaign.

3) *Attack Pattern Instantiation:* In Section III-B, we discuss the creation of attack templates through the generalization of system entities. While instantiating an attack pattern template, these generalized system entities are replaced by appropriate and meaningful artifacts (or a hyponym) to form a validated attack pattern. This approach enables the generation of a wide range of attack instances.

There are two methods for collecting the hyponyms of generalized system entities. The first method involves using artifacts collected from real-world data, such as malware names, attack-related file names, and registry paths. To gather these artifacts, we leveraged VX-Underground [22] and VirusTotal [23]. VX-Underground provides approximately 10,000 malware hash values associated with APT campaigns collected over the past five years. Each hash was subsequently queried on VirusTotal to extract relevant artifacts from the behavioral analysis reports. In total, 157,806 artifacts were obtained and mapped to the generalized system entities defined in the attack templates for subsequent instantiation. For example, by querying VirusTotal using a sample hash value 78a3e4702d9fc13b2ef917211cd65b44, we

Algorithm 2: Synthetic Audit Log Generation.

Input: Lifecycle $\{LC\}$, Attack pattern instances $\{In(TQ)\}$, Benign audit log $\{BA\}$
Output: Synthetic audit events SA

```

1: while while  $LC_i$  in  $LC$  exists do
2:   for each  $stage$  in  $LC_i$  do
3:     // Include an attack pattern instance  $In(TQ)$  in the
        $stage$ . Line 8 of Table II.
4:      $MA = BE + In(TQ) + MA \mid BE + In(TQ)$ 
5:     where  $In(TQ)$  is an instantiated attack template,
       as Line 10 of Table II.  $BE$  is a short list of benign
       events.
6:   end for
7: end while //Obtain all malicious audit logs  $\{MA\}$ 
8:
9: Initialize  $SA$  with given background events.
10: for each  $BA_i$  in  $\{BA\}$  do
11:   // Perform  $C_b(SA, BA_i)$ , as Line 1 of Table II.
12:   for each  $evt$  in  $\{BA_i\}$  do
13:      $evt.start = BA_i.start + evt.relative$ 
14:   end for // Insert event  $evt$  to synthetic audit log.
15:    $SA = SA \cup BA_i$ 
16: end for // Insert benign audit log  $BE_i$ .
17: for each  $MA_i$  in  $\{MA\}$  do
18:   // Perform  $C_m(SA, MA)$ , as Line 1 of Table II.
19:   for each  $In(TQ_j)$  in  $\{MA_i\}$  do
20:     for each  $evt$  in  $\{In(TQ_j)\}$  do
21:        $evt.start = In(TQ_{j-1}).end +$ 
          $In(TQ_j).lapse + evt.relative$ 
22:     end for // Insert event  $evt$  to synthetic audit log.
23:      $SA = SA \cup In(TQ_j)$ 
24:   end for // Insert technique  $In(TQ)$  to synthetic log.
25: end for // Insert malicious audit log  $MA$  to synthetic
       audit log.

```

can extract a file `%APPDATA%\... \Startup\twainResolver.lnk`. The dropped file, *twainResolver.lnk*, is classified under the system entity type *file*, as indicated in the ‘file’ section of the VirusTotal report. It is then recognized as ‘file.short’ based on regular expression matching, and a shortcut file in the Windows startup folder to implement an attack behavior such as *T1547.009 Shortcut Modification*.

The second source of artifacts, such as usernames and process IDs, which are indicated with * Table I, is generated using the Faker library in Python [21]. Faker efficiently generates realistic-looking random data that mimics real-world information. These random artifacts are used to populate attack instances within the templates. Once attack patterns are generated for all stages as a chronological sequence of labeled events, the prerequisites for every attack pattern must be satisfied to validate the attack sequence, which is then considered an APT campaign.

D. Synthetic Audit Log Generation

A synthetic audit log comprises several benign and APT campaign audit logs, with their composition methods detailed in

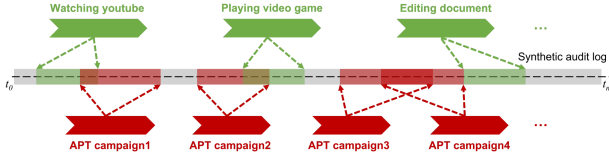


Fig. 6. Composition of synthetic audit log.

Algorithm 2. A benign audit log is recorded from normal software activities, such as web browsing, Windows Office usage, file downloads, etc., and is represented as a sequence of events, as defined in lines 2 and 3 of Table II. Each benign audit log includes a start time (BA.start) and log length (BA.length), with each event having a relative time (evt.relative) to the log start time. The notation $C_b(SA, BA)$ in the first line of Table II denotes the composition of a benign audit log BA into a synthetic audit log SA . The composition involves setting the starting time of the first event to the specified time of the benign audit log, and adjusting the start times of all other events by adding the log start time to their relative times. The procedure is described in lines 9 to 14 of Algorithm 2.

For the composition of an APT campaign audit log into a synthetic audit log, denoted as $C_m(SA, MA)$ in the first line of Table II, it merges each pair of benign events (BE) and attack technique events ($ln(TQ)$) sequentially until all pairs are combined. A BE is usually a short sequence of benign events that occurs before the particular Technique ($ln(TQ)$). The composition of BE follows the same method as that for BA . The composition of TQ , which refers to the audit log events representing a particular Technique, provides flexibility in both the start time and duration. This means that a Technique audit log can begin at any time and span any length. The start time of a Technique audit log is set to the end time of the preceding Technique plus the lapse time $ln(TQ).lapse$, which can be specified by the user or determined randomly by SAGA. The procedure is described in lines 15 to 22 of Algorithm 2.

Fig. 6 depicts a composition of 3 benign audit logs and 4 APT audit logs. Note that the logs may overlap and the APT audit logs can be extended for any duration. SAGA explicitly leverages process identifiers, parent-child process relationships, and event timestamps to preserve the underlying process structure, temporal ordering, and causal dependencies that are critical for contextual consistency in audit logs. To avoid unrealistic combinations of events, SAGA maintains semantic coherence at both the process and artifact levels. For example, when two events involve shared entities, such as a file, registry key, or named pipe, SAGA preserves these dependencies to ensure that the resulting logs reflect plausible and continuous behavior. As illustrated in Section II-C, a user downloads a malicious attachment IOC_09_11.rar via a browser ($T1566.001$: *Spearphishing Attachment*), and subsequently opens it using Microsoft Word ($T1204.002$: *Malicious File Execution*). Although these steps span separate processes, their interaction through a shared artifact forms a coherent and realistic attack sequence. In this way, SAGA ensures that injected attack behaviors are not only chronologically plausible but also contextually consistent with

normal system operations, mitigating the risk of generating disjointed or unrealistic event flows.

There are still tasks that need to be addressed, such as establishing process relationships. For example, in an audit log, the ProcMon operation *ProcessCreate* (as shown in Fig. 1) creates a new process, and their parent-child relationship must be preserved. SAGA treats artifacts with the same descriptor within a single attack pattern template as belonging to the same process. Otherwise, operations from different templates are considered independent processes.

IV. USEFULNESS STUDY OF SYNTHETIC AUDIT LOG

In this section, we evaluate the usefulness of synthetic audit logs for various detection methods in different APT attack scenarios. First, we show the variety of synthetic audit logs that SAGA can produce, from those that mimic real-world APT attacks (denoted as the C Group) to randomly generated logs (denoted as the G group), and finally to composite both groups (denoted as the M group). Next, we assess whether state-of-the-art audit log-based and provenance-based malicious detection methods can effectively use the synthetic audit log as their target datasets. In addition, we discern the performance and operation differences of these methods when applying their original datasets versus various synthetic data logs. Note that the objective of this study is not to identify the best-performing methods or models, as each approach has its own specific focus, design considerations, and inherent limitations.

This section begins by outlining the settings for the empirical study, including the synthesized datasets, labeling, metrics, and machine hardware and software configurations. Our analysis focuses on three sets of empirical studies to demonstrate the usefulness of the synthesized attack datasets: 1) assessing the effectiveness of APT intrusion detection by, 2) evaluating the accuracy of Technique hunting, and 3) gauging the effectiveness of campaign attribution. Finally, we will conduct a case study to evaluate the effect of model training using synthetic audit logs to test the model on an unseen dataset.

A. Evaluation Settings

Datasets: The experiments utilize synthetic audit log datasets generated by SAGA (hereafter referred to as the SAGA dataset), which is publicly available at [24]. A synthetic audit log, as described in Section III, includes both benign behaviors and malicious APT attack scenarios. Similarly to previous research [5], [25], the benign audit logs cover a variety of everyday activities, such as watching YouTube videos, downloading and compiling source code from GitHub, writing and sending emails, running Python programs, playing video games, reading news articles, and composing Word documents. This diversity ensures that the synthetic audit logs accurately reflect typical user behavior. For malicious events, we prepared 169 abilities executed on the MITRE CALDERA [19] red-team emulator to generate labeled malicious audit events. These events were then processed into attack pattern templates for further synthetic audit log generation. Each of the 169 attack pattern templates was instantiated with artifact replacements, as described in Section III-B, to form

the datasets used in the empirical study. The training dataset for Technique hunting consists of 16,900 attack patterns, each of which is an instantiation of a attack pattern template. This dataset is split into training, validation, and test sets in an 8:1:1 ratio.

Given the limited availability of audit logs labeled with Techniques, we used the attack pattern templates to create APT campaigns. These scenarios include eight APT attack campaigns (“known” group) based on cyber threat intelligence reports from real-world APT campaigns, including Higaia [26], admin338 [27], APT28 [17], FIN7 [28], CobaltGroup [29], Gamaredon [30], Patchwork [31], and GorgonGroup [32]. Moreover, using SAGA random generation capability, we produced 20 APT campaigns (“random generated group”). From these 28 APT campaigns, we randomly selected 3 to create new composite scenarios involving multiple APT campaigns targeting a single victim host, resulting in 10 composite campaigns (“composite” group). To demonstrate the versatility and flexibility of SAGA, each of the above APT campaigns was generated over three different time periods: 15 minutes, 1 h, and 1 d.

Labeling: For each generated APT campaign, SAGA labels malicious events with APT stages, Techniques, abilities, manipulated system entities, and other operation-related attributes. In addition, it includes threat actors, when applicable. Malicious audit events are labeled using the BIO2 scheme [33] covering 7 attack stages, 80 Techniques, and 169 abilities, as shown in Table S1 of Supplementary Material A.

Evaluation Metrics: We adopt three evaluation metrics for different tasks: APT intrusion detection, Technique hunting, and APT campaign attribution. The APT intrusion detection task aims to identify any APT intrusions or anomalies within the dataset under examination. Technique hunting seeks to identify Techniques violated based on MITRE ATT&CK definitions. APT campaign attribution aims to determine the most probable APT campaign present in the dataset. To evaluate performance, we use standard classification metrics: precision (P), recall (R), F1 score (F1), and accuracy (ACC).

For APT intrusion detection, we frame it as a binary classification problem, determining whether a malicious event exists in the audit log. Since different target systems or methods apply their own tests to detect APT intrusion on an event, a graph, or a time window interval, we adapt to their respective evaluation designs to assess performance. Technique hunting is treated as a multi-class classification task, where each classifier determines whether a specific Technique has been employed in one or more events. For APT campaign attribution, we analyze the Techniques identified from the Technique hunting task to determine the most likely APT campaign for the audit log. We rank the ground truth against the predictions and employ a Top-K measurement to evaluate the performance of the APT campaign attribution.

Established Baselines: For APT intrusion detection, we demonstrate the usefulness of generated attack datasets using three types of systems: event-based (AirTag [6]), graph-based (Unicorn [34]), and time window-based (KAiOS [7]) intrusion detection systems. The reasons for choosing the three systems are not only that they are established baselines, but also that their

TABLE III
COMPUTATION OVERHEAD OF AUDIT LOG GENERATION

	Synthesizing APT Campaign	Audit Log Composition		
		15 min	1 hour	1 day
Memory (MB)	46.95	2644.05	4407.96	41078.54
Time (Sec)	54.87	47.07	76.61	322.84

codes are publicly available and the systems can be installed and executed without issues.

For Technique hunting, we used the Sigma rules [35], an open signature-based rulebase contributed by numerous security practitioners, and SFM [36], a deep learning approach that provides Technique hunting functionality. We selected Sigma as a baseline because it is a widely adopted, community-driven detection framework, actively maintained by practitioners worldwide. As noted by [37], Sigma has also been shown to align closely with several commercial rulesets, making it a meaningful and representative point of comparison. A major portion of these rules align with the MITRE ATT&CK framework with 70 rules mapped to the Techniques identified in this study. SFM integrates three key technologies: anomaly detection to mitigate the impact of data imbalance, a multi-class classifier for Technique hunting, and subgraph matching for APT campaign attribution. SFM was selected because it was one of the few systems specifically designed to address both the Technique hunting task and APT campaign attribution, making it particularly suited for the evaluation of complex attack scenarios. All models and methods were installed with configurations as close as possible to those described in their original publications. Code modifications were only made to ensure compatibility with the structure of the synthetic audit logs.

Computation Configurations: For empirical studies, we used a Windows 10 operating system with 8 GB of memory with an Intel i9-10900F CPU as our victim system. For defender model development, we employed AMD EPYC 7282 16-Core Processor CPU running Ubuntu 20.04 system with 1 TB of memory and two NVIDIA A100 80 GB GPUs. Neural network training was performed using PyTorch 1.13.1 and Python 3.10. The computational overhead of generating malicious audit events and audit log composition was measured and listed in Table III.

B. APT Dataset Generation

In this section, we provide a more detailed discussion on the considerations for generating the APT dataset, focusing primarily on generation guidance, attack scenarios, and duration.

Generation Guidance: SAGA offers flexibility in generating APT campaigns, based on user specifications, randomly, or through a combination of both. For example, SAGA generates eight known APT campaigns (denoted from C1 to C8) following user specifications, with their lifecycles and Techniques presented in Table IV. To further illustrate SAGA’s versatility, we also used its random generation capabilities to create 20 additional campaigns (denoted from G1 to G20), showcasing its capacity to create varied attack scenarios. The lifecycles and Techniques of these randomly generated campaigns are presented in Table S2 of Supplementary Material A, while

TABLE IV
APT ATTACK SCENARIOS BASED ON CYBER THREAT INTELLIGENCE REPORTS

ID	Simulated Campaign	Attack Stage	Techniques
C1	Higaisa [26]	{1,2,6,4,4,6,6}	PA, MFE, RK3, SID, SNCD3, MTOS1, ST2
C2	admin338 [27]	{1,2,4,4,4,4,4,4}	PA, MFE, LA1, FDD1, LG1, SNCD6, SNCOD1, SSD3
C3	APT28 [17]	{1,2,2,4,4,7}	PA, WP, MFE, SID, DLS, EWS
C4	FIN7 [28]	{1,2,6,6}	PA, ITT3, RK4, ST1
C5	CobaltGroup [29]	{1,2,4}	PA, RAS1, NSD
C6	Gamaredon [30]	{1,2,2,6,6,4,4,6,7}	PA, WP, MFE, MR3, RK2, WMI5, SID, ST2, DF2
C7	Patchwork [31]	{1,2,3,4,4,4,6,5}	PA, PS2, BUAC, DLS, UD2, SD1, RK3, RDP
C8	GorgonGroup [32]	{1,2,6,6,6,6,6}	PA, PS1, PEI, RK3, SM1, DMT, HW

Lifecycle is indexed by number. Techniques include PA = phishing Attachment, MFE = Malicious File Execution, RK = Registry Run Keys, SID = System Information Discovery, SNCD = System Network Configuration Discovery, MTOS = Masquerade Task or Service, ST = Scheduled Task, LA = Local Account, FDD = File and Directory Discovery, LG = Local Groups, SNCOD = System Network Connections Discovery, SSD = System Service Discovery, WP = Web Protocols, EWS = Exfiltration Over Web Service, ITT = Ingress Tool Transfer, RAS = Remote Access Software, NSD = Network Service Discovery, MR = Modify Registry, WMI = Windows Management Instrumentation, DF = Defacement, PS = PowerShell, BUAC = Bypass User Account Control, DLS = Data from Local System, UD = System Owner/User Discovery, SD = Security Software Discovery, RDP = Remote Desktop Protocol, PEI = Portable Executable Injection, SM = Shortcut Modification, DMT = Disable or Modify Tools, HW = Hidden Window. The subsequent number of a technique represents a distinct ability used to implement that technique.

TABLE V
STATISTICS FOR THE THREE TIME DURATIONS IN THE EIGHT APT ATTACKS

ID	# Malicious event	15 min			1 hour			1 day		
		size(MB)	# entity	# event	size(MB)	# entity	# event	size(MB)	# entity	# event
C1	30	303.27	31,193	609,394	1,344.68	255,180	2,400,089	7,423.86	696,844	14,647,429
C2	53	472.79	34,931	956,654	1,304.79	138,851	2,609,138	7,394.81	747,445	14,548,395
C3	14,137	684.40	182,004	1,209,895	179.00	33,546	360,313	7,447.71	709,641	14,682,885
C4	22	1,161.21	276,956	2,072,382	137.75	14,912	290,962	7,414.71	696,483	14,631,215
C5	1,133	483.48	36,466	969,542	730.36	73,220	1,475,802	6,622.55	667,110	12,914,561
C6	59	225.76	31,494	453,915	199.48	22,316	399,084	8,002.90	741,686	15,615,065
C7	14,124	83.51	29,655	163,487	800.95	131,125	1,623,759	6,415.17	669,041	12,558,106
C8	50	494.16	53,386	850,419	153.08	18,312	316,953	7,375.18	747,088	14,509,490
Mean		488.57	84,511	910,711	606.26	85,933	1,184,513	7,262.11	709,417	14,263,393

the detailed statistics, including sizes, number of entities, and number of events, are provided in Table S3 also in Supplementary Material A. This flexibility highlights the scalability and adaptability of our approach in simulating a wide range of cyberattack scenarios, enabling the creation of customized and diverse testing environments to thoroughly evaluate detection methods.

Attack Scenario: We introduce two variations in attack scenarios: one involving a single campaign targeting the system and another featuring three simultaneous campaigns. This variation in attack composition allows for a comprehensive assessment of detection methods in more complex, multi-campaign situations. For instance, the M1 attack scenario consists of three separate attacks, Higaisa, G1, and admin338, each employing distinct Techniques and targeting the same compromised host. Additional details on the ten generated composite campaigns can be found in Table S4 in Supplementary Material A, due to the constraints of the manuscript space.

Attack Duration: To account for the variability in the duration of the attack, we generated synthetic audit logs with durations of 15 minutes, 1 h, and 1 d. Although SAGA can create logs of any length, we limited the maximum duration to 1 d in this study, considering the computational overhead of the intrusion detection systems discussed in Section IV-C Table V, Table S3 and Table S4 in Supplementary Material A, presents statistical descriptions of logs for each duration. As shown in Table V, the average number of events increases with log length: 910,711 for 15-minute audit logs, 1,184,512.5 for 1-hour audit logs, and

14,263,393.25 for 1-day audit logs. This variability reflects the evolving nature of APT attacks, which are not constrained by fixed timeframes, highlighting SAGA ability to generate audit logs of various durations. It also allows for a more thorough examination of how different detection methods perform over varying time intervals.

C. APT Intrusion Detection

APT intrusion detection is defined as the task of identifying potential APT attacks within an audit log, as established in previous studies [6], [7], [34]. To demonstrate the usefulness of SAGA datasets in supporting APT intrusion detection, we present the detection results using their own evaluation schemes of various methods and then discuss the causes of performance difference, considering the following factors: test granularity, Technique used, attack scenario, and duration. Table VI presents the performance of the three intrusion detection systems, AirTag (event-based), Kairos (time window-based) and Unicorn (graph-based) across various SAGA datasets. More details can be found in Tables S5, S6, and S7 of Supplementary Material B. Performance data for E3-CADETS, E3-THEIA, and E3-ClearScore were copied directly from their respective original papers. Unicorn, which analyzes the entire provenance graph to detect anomalies, generally performed well in both precision and recall. AirTag, on the other hand, which identifies intrusions by analyzing individual events, struggled with high rates of false positives and false negatives. When exposed to SAGA's broader technique

TABLE VI
AVERAGE PERFORMANCE OF THE INTRUSION DETECTION SYSTEMS

Campaign	Precision			Recall			F1			Accuracy			AUC
	AirTag	Kairos	Unicorn	AirTag	Kairos	Unicorn	AirTag	Kairos	Unicorn	AirTag	Kairos	Unicorn	Kairos
E3-CADETS	-	100.00%	98.00%	-	100.00%	100.00%	-	-	99.00%	-	100.00%	99.00%	-
E3-THEIA	-	100.00%	100.00%	-	100.00%	100.00%	-	-	100.00%	-	100.00%	100.00%	-
E3-ClearScope	-	100.00%	98.00%	-	100.00%	100.00%	-	-	99.00%	-	100.00%	99.00%	-
C_15min	avg	4.04%	49.73%	100.00%	70.64%	97.08%	94.25%	6.17%	58.87%	96.52%	80.07%	57.03%	94.25%
	std	11.21%	35.30%	0.00%	33.71%	4.25%	5.53%	17.00%	33.81%	3.63%	6.77%	29.23%	5.53%
G_15min	avg	0.09%	43.24%	99.90%	61.24%	91.39%	97.38%	0.18%	53.71%	98.29%	80.23%	57.32%	97.38%
	std	0.13%	27.46%	0.31%	27.73%	12.38%	4.86%	0.26%	25.35%	3.39%	12.95%	22.96%	4.86%
M_15min	avg	6.41%	48.55%	94.60%	62.19%	82.32%	78.88%	9.47%	55.44%	82.00%	77.53%	49.65%	78.88%
	std	14.49%	27.84%	3.89%	28.93%	9.86%	5.46%	20.50%	25.74%	5.44%	7.92%	20.60%	4.96%
C_1hour	avg	2.76%	40.64%	83.55%	76.65%	85.95%	95.13%	4.84%	52.83%	83.55%	76.26%	55.27%	85.20%
	std	6.01%	24.39%	11.66%	32.45%	29.63%	4.22%	10.33%	26.60%	11.66%	13.28%	15.14%	10.85%
G_1hour	avg	0.04%	38.91%	85.20%	60.49%	96.47%	96.25%	0.09%	53.37%	85.20%	77.36%	45.28%	87.13%
	std	0.05%	16.79%	11.69%	28.83%	6.79%	5.42%	0.10%	18.04%	11.69%	13.25%	14.58%	11.49%
M_1hour	avg	1.82%	33.63%	87.92%	75.68%	93.09%	97.90%	3.42%	47.32%	87.92%	78.49%	46.00%	90.06%
	std	3.03%	17.80%	9.86%	17.94%	8.63%	4.28%	5.66%	20.51%	9.86%	13.15%	16.02%	9.44%

C, G, and M denote known, randomly generated, and composite campaigns.

coverage, multi-stage campaign structure, and temporal variation, AirTag, a one-class SVM anomaly detection model built on a customized BERT embedding, struggles to find a stable decision boundary across such diverse contexts. Kairos was designed with specific assumptions about event sequence timing, which may not align with the more flexible and richly structured scenarios generated by SAGA. By grouping events within time windows, Kairos tended to reduce both false positives and false negatives, resulting in relatively low precision but higher recall.

Test granularity: Test granularity refers to the unit of target tested in a detection method. For example, the text granularity of AirTag is based on individual events. In a test using the APT28 campaign dataset (15 minutes), AirTag generated 301,069 false positives and missed 14,098 malicious events out of 1,209,895 total events. On the other hand, Kairos, which uses time window as its test granularity, incorrectly flagged 42 benign windows and missed just one malicious window out of 67 total windows. Table VI presents the performance of the three detection systems, each using its own scoring method (i.e., a formula to calculate the performance metrics). This highlights the significant influence of the test granularity and scoring method on the reported results, making direct comparisons between performance data difficult and, at times, not meaningful. Since SAGA provides labels for each event, its datasets are versatile enough to support various test granularity and scoring methods.

Technique impact: The generated campaigns (G) feature a greater number and diversity of Techniques compared to the synthetic known campaigns (C). To assess the impact of varying the number and types of Techniques used in different campaigns, we provide details of the audit logs for the known campaigns in Table IV, and for the generated campaigns in Table S2 of Supplementary Material A. As shown, the audit logs for the generated campaigns contain more and a broader range of Techniques than those in the synthetic known campaigns. Table VI presents the average performance of each detection system, showing that the C group consistently outperforms the G group. This decline in performance is directly correlated with the increasing number and diversity of Techniques.

Attack Scenario Impact: SAGA is capable of generating synthetic audit logs containing multiple APT campaigns, referred to as composite campaigns (M group), designed to reflect complex real-world scenarios. Details of these composite campaigns, labeled M1 to M10, are provided in Table S7 in the Supplementary Material B. As the complexity of the M group audit logs increases, the detection performance gradually declines (e.g., from C_15 min to M_15 min), as shown in Table VI.

Duration Impact: To demonstrate its ability to generate synthetic audit logs of varying length, 15 minute and 1 h attack scenarios were generated to detect APT intrusions. An APT attack typically employs a “low-and-slow” approach, maintaining a prolonged presence on the target machine, which makes the intrusion difficult to detect. As shown in Table VI, the detection performance gradually decreases with longer durations for the same context.

In summary, synthetic audit logs can be tailored in terms of duration, the number and type of embedded APT campaigns, and the types of APT campaigns. As the complexity of these scenarios increases, the performance of detection methods tends to degrade.

D. Evaluation on Technique Hunting

To demonstrate the practical utility of fine-grained Technique labeling, Technique hunting is the task of identifying potential threats, specifically Techniques, as defined by the MITRE ATT&CK framework. SAGA follows Algorithm 2 to generate attack campaigns, where each event is labeled with its corresponding Technique identifier. This structured synthetic audit log allows detection methods to effectively perform Technique hunting. The approaches are used for Technique hunting; one is SFM and the other is using Sigma rules. The performance results of Technique hunting for each APT attack campaign are reported in Table VII that details are given in Table S9 in the Supplementary Material C. The results indicate significant performance distinctions between SFM and Sigma rules, regardless of the generation method, attack scenario, or duration. SFM is capable of recognizing attack patterns by learning from synthetic data,

TABLE VII
THE PERFORMANCE OF TECHNIQUE HUNTING

Campaign	Precision		Recall		F1	
	Sigma	SFM	Sigma	SFM	Sigma	SFM
C_15min	5.58%	58.67%	8.50%	81.20%	2.34%	63.86%
G_15min	30.44%	53.65%	7.49%	83.79%	9.04%	58.85%
M_15min	4.61%	65.27%	7.46%	87.39%	2.16%	68.50%
C_1hour	9.35%	57.35%	11.13%	79.63%	2.67%	62.72%
G_1hour	45.54%	39.98%	8.65%	84.38%	11.92%	47.84%
M_1hour	4.62%	47.43%	7.46%	87.73%	2.20%	56.27%
C_1day	0.38%	5.04%	8.63%	77.22%	0.58%	9.21%
G_1day	6.00%	2.39%	10.20%	83.71%	6.28%	4.51%
M_1day	0.83%	6.87%	7.05%	88.44%	1.03%	11.88%

whereas Sigma rules are limited to detecting attack patterns that match pre-defined signatures. This underscores the importance of the generated datasets in facilitating the advancement of learning-based model methodologies.

However, both methods showed significant performance fluctuations across various attack scenarios. For example, the average standard deviations of precision during the 15-min scenarios were 33.92% and 32.25% for SFM and Sigma rules, respectively. Several factors contributed to this variability. First, there was a high incidence of false positives associated with specific attack behaviors. Second, the substantial imbalance between the number of malicious events and the overwhelming prevalence of benign activities skewed the evaluation of attack behaviors. Lastly, benign activities and malicious events were intertwined and exhibited different patterns in various datasets, complicating the recognition of attack patterns.

For instance, recognizing T1105 Ingress Tool Transfer (ITT3) requires identifying the following sequence of audit events within interleaved benign event sequences: *<browser.exe, TCP connect, XXX.com>*, *<browser.exe, FileCreate, payload.exe>*, and *<xxx, Process Create, payload.exe>*. This sequence indicates that a malicious payload was downloaded from an external C2 server and executed on a compromised machine. These malicious events are causally correlated and in audit log, they were intertwined with benign and background events, making the recognition of this Technique particularly challenging.

On the other hand, while Sigma rules perform efficiently, they are based on contributions from human experts and may not encompass all attack behaviors, such as T1055.002 Portable Executable Injection (PEI) and T1491 Defacement (DF2) identified in this empirical study. As a result, certain malicious behaviors went undetected in the APT campaigns C-6, C-8, G-11, and G-13. These findings address the limitations of manually crafted, signature-based detection methods and highlight the importance of datasets like those generated by SAGA, which provide comprehensive logs from red-team emulation campaigns. Such datasets support the development and evaluation of both rule-based detection systems and learning-based models in a realistic environment.

E. APT Campaign Detection Evaluation

APT campaign detection involves identifying a potential APT campaign embedded in an audit log, which is characterized by

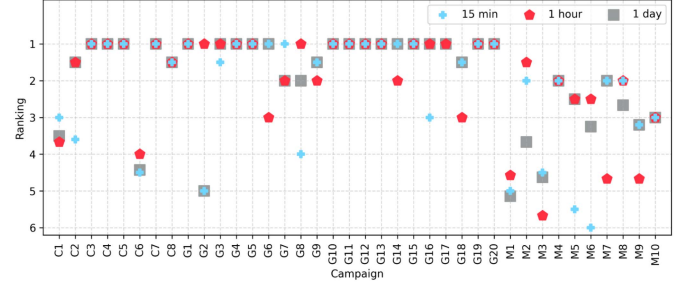


Fig. 7. The top k ranking results of APT campaign detection by SFM.

a series of coordinated attack patterns. In line with the MITRE ATT&CK framework, an APT campaign can be described as a sequence of Techniques. SFM leverages this structure to detect the most likely APT campaign by comparing the discovered Techniques (G_q) against a pool of known APT campaigns (G_c). In this empirical study, G_c includes 8 known campaigns and 30 randomly generated APT campaigns. Fig. 7 illustrates the ranking of each SFM-identified campaign over three different durations, 15 minutes, 1 h, and 1 d. SFM successfully identified most campaigns within the top 3 rankings across all three time durations: 74% for 15 minutes, 84% for 1 h, and 79% for 1 d. Notably, 50% of the campaigns were ranked first in both 1 h and 1 d scenarios. One reason for this success could be that the temporal causal relationships is established among the Techniques discovered by SFM, which effectively distinguishes the most probable campaign from others. Furthermore, SFM demonstrated resilience to discrepancies between the discovered ability graph G_q and the campaign Techniques G_c . However, the M group, consisting of multiple campaigns in a single scenario, posed a greater challenge compared to the groups of single campaigns. In addition, the results further suggest that as the duration of audit log grows, the complexity of identifying APT campaigns also increases. In conclusion, this evaluation highlights the potential of SAGA in effectively supporting the detection of APT campaigns.

F. A Case Study of Unseen APT Campaign

In this section, we investigate the effectiveness of SAGA synthetic audit log datasets in training an APT campaign model to be applied to previously unseen APT campaigns, even when the campaign audit logs follow a different format and organization. Specifically, SFM was trained using SAGA datasets and tested on the DARPA TC E3 Fivedirection dataset to detect attack behaviors (i.e., Techniques) within the logs.

The E3 Fivedirection dataset includes a complex attack scenario (4.4 Phishing Email with Excel Macro [38]). As illustrated in Fig. 8, the attacker used a phishing attachment to lure the victim into downloading and executing malware that triggered subsequent malicious activities, including establishing a remote command shell and gathering various system information. From the detection results, it shows that SFM was able to successfully identify part of the attack sequence. For example, during the initial compromise phase, the SFM successfully

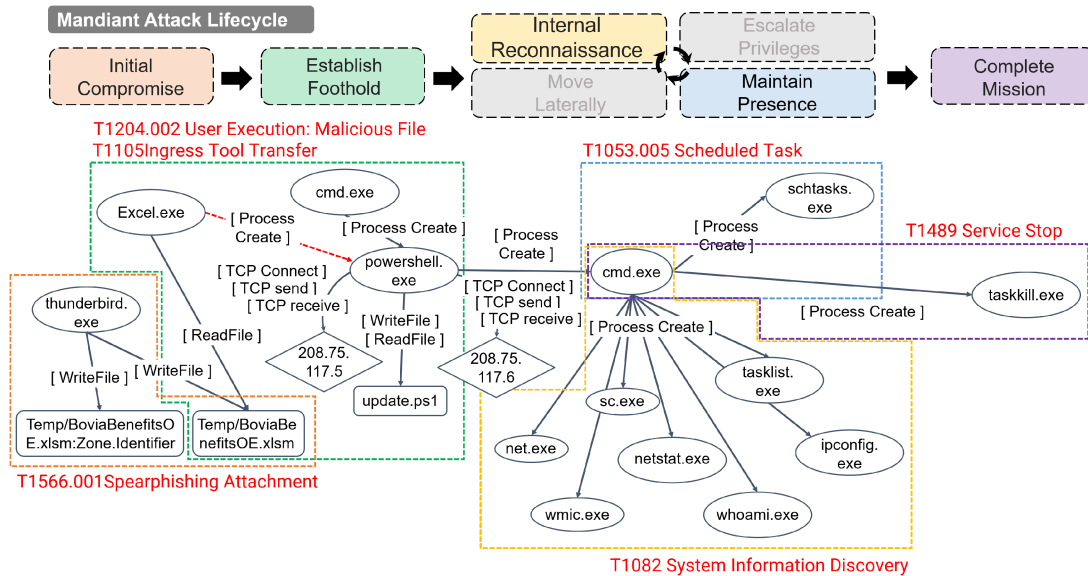


Fig. 8. The DARPA TC E3 Fivedirection attack graph.

flagged the system events corresponding to the victim downloading the phishing attachment, $\langle \text{thunderbird.exe}, \text{WriteFile}, \text{BoviaBenefitsOE.xlsm} \rangle$, as T1566.001 Spearphishing Attachment. Additionally, during the Establish Foothold phase, the system events corresponding to the execution of the phishing attachment, $\langle \text{Excel.exe}, \text{CreateFile}, \text{BoviaBenefitsOE.xlsm} \rangle$, were also correctly flagged as T1204.002 User Execution: Malicious File.

However, some attack behaviors were not correctly identified by SFM. For example, during the Establish Foothold phase, when the attacker used PowerShell to download a malicious payload (*update.ps1*) from a C2 server, SFM successfully marked the system events related to the download activity, $\langle \text{powershell.exe}, \text{CreateFile}, \text{update.ps1} \rangle$, as T1105 Ingress Tool Transfer, but it failed to include the system events, PowerShell connection to the C2 server, $\langle \text{powershell.exe}, \text{TCP connect}, 208.75.117.5 \rangle$ in T1105. This limitation could be attributed to gaps in the SAGA dataset's representation of certain attack behaviors, or SFM's difficulty in detecting subtle variations in event sequences.

However, these results highlight the practical value of the SAGA dataset in supporting practical cybersecurity applications and providing a solid foundation for future improvements to dataset generation strategies. They also underscore the need for further improvements in APT campaign detection models to enhance their capability and accuracy.

V. RELATED WORK

Existing Benchmarks for Host-based Intrusion Detection: As noted in the survey by [3], the majority of studies on host-based intrusion detection have relied on self-constructed attack scenarios for evaluation. Some works have utilized existing public datasets such as CERT's dataset [39], LANL's

dataset [10], StreamSpot [25] ADFA [40], and DARPA Transparent Computing (TC) [8], as well as OpTC [9]. CERT's dataset provides synthetic background data alongside malicious operations, such as logon and email activities, carried out by multiple threat actors across five distinct attack scenarios. LANL's dataset consists of 58 consecutive days of event data collected from LANL's corporate computer network, including logon events, process events, DNS lookups, network flow events, and a set of well-defined red teaming events representing malicious activities. StreamSpot includes one attack activity—a drive-by download initiated by visiting a malicious URL that gains root access to the visiting host—along with five benign activities forming three scenarios. However, these datasets primarily focus on event action types (i.e., operations) and timestamps, lacking critical details regarding the underlying system entities involved in the audit events. This absence limits their ability to clearly describe the full context of each event, which is essential for comprehensive attack pattern analysis and detection.

ADFA comprises three datasets: ADFA-LD (Linux), ADFA-WD (Windows), and ADFA-WD-SAA (Windows for Stealth Attacks). Among these, ADFA-WD is the most relevant to SAGA. However, it only contains DLL traces of processes collected by Procmon and includes twelve zero-day attacks executed on a host.

DARPA's TC program included five engagements and the OpTC dataset. The E3 and E5 engagements, featuring two and eight multi-stage APTs respectively, are widely used for intrusion detection, alongside benign background activity. The OpTC dataset, created to assess scalability, simulated a three-day APT on a 1,000-host test network during a two-week experiment. However, these datasets lack thorough documentation and labels, which limits their usefulness for precisely evaluating attack behaviors and audit events.

TABLE VIII
COMPARISONS WITH SYNTHETIC APT DATASET GENERATION

Log Generation	DAPT2020 [42]	PWNJUTSU [43]	Unraveled [44]	LADEMU [45]	SAGA
Duration	3 months	-	6 weeks	1 hour	Configurable
# Attackers	1	22	3	1	Configurable
Attack Stage	4	5	5	4	6
# Attacks	16	13	15	79	197 (extendable)
Label	Stage, Attack Behavior	Stage, Technique	Activity, Stage, Defender Response, Signature	Tactic, Technique	Stage, Technique, Ability, Campaign
Alignment to MITRE ATT&CK	N	Y	Y	Y	Y
Threat Model	Human Experts	Human Experts	Employee Behavior Generation	Red team emulator (Caldera)	Red team emulator (Caldera)
Automation	●	●	●	● (PoC)	●

Synthetic APT Dataset Generation: While there are some publicly available attack datasets, few research focus on constructing datasets for simulating multi-stage APT attacks [41], [42], [43], [44]. To steal organization data, DAPT2020 [41] imitated the exploited web vulnerability in a production network, and Unraveled [43] created skeletons of user profiles to emulate user behavior based on predefined job pipelines. These datasets encompass network traffic and multi-host log information, covering limited attacks (i.e., 10 and 15) within attack stages over a fixed collection duration (i.e., 1 week or 6 weeks). In contrast, our generated APT dataset includes and labels 7 attack stages, 80 techniques, and 169 abilities, offering *configurable* attack patterns and collection duration on single-host audit events. This flexibility allows for more detailed and customizable simulation of attack behaviors.

PWNJUTSU [42] and LADEMU [44] considered adversaries' TTPs, defining the operational semantics of Techniques as specifications for procedures. PWNJUTSU utilized a semi-automated approach where each attack step was crafted by professional human Red Teamers, providing a highly skilled yet partially manual campaign construction process. In contrast, LADEMU offered a proof-of-concept (PoC) implementation using the red-team emulator Caldera and demonstrated the use of a single attack scenario, APT29. Our study introduces a workflow of campaign generation with event logs, enabling fully automated APT simulations based on TTPs specified in the MITRE ATT&CK framework. This approach eliminates the need for manual intervention, ensuring more consistent and scalable attack simulations. Table VIII contrasts SAGA with previous framework and highlights SAGA's unique advantages, particularly its configurability (in terms of duration, techniques, and campaign design), fine-grained labeling, and high degree of automation.

Host-based Provenance analysis: Learning-based provenance analysis methods have shown notable improvements in the detection of intrusions and have accelerated the investigative process of various attacks [3], [5], [6], [7], [34], [45], [46], [47].

These learning-based models employ a range of representation learning techniques to encode provenance graphs, enabling effective anomaly detection. Examples include graph

representation methods used in Unicorn [34], Kairos [7], and MAGIC [46]; translation-based embedding techniques in WATSON [48] and ShadeWatcher [45]; and language models in AirTag [6] and DrSec [47]. Our research demonstrates that applying event-based [6] and graph-based intrusion detection [7], [34] to our synthesized campaign datasets results in effective detection of intrusion events.

Technique Hunting: HOLMES [49] generates detection signals by matching 16 predefined attack patterns and constructing a high-level scenario graph embedded with TTPs to indicate APT campaign activities. RapSheet [50] creates a tactical provenance graph using 67 rules within a commercial EDR tool, aligning it with MITRE ATT&CK. KRYSTAL [51] leverages Sigma rules [52] to detect known attack patterns, converting them into SPARQL queries for threat detection. Although these rule-based approaches are effective, their manual creation and upkeep are resource-intensive. Our study addresses this limitation by introducing a flexible approach to APT dataset generation using a red-team emulator for audit event collection, enabling a learning-based methodology. Additionally, we propose a straightforward method (SFM) for identifying attack patterns, providing a baseline for future research.

VI. DISCUSSION AND CONCLUSION

This study presents a generative APT campaign framework, SAGA, designed to synthesize synthetic audit logs for nearly any configuration. These logs support the training of various machine learning-based APT-related detection machine learning models and facilitate the evaluation of various APT-related detection methods. To demonstrate the usefulness of SAGA's synthetic datasets, a collection of empirical studies were conducted across multiple established baselines for APT intrusion detection, Technique hunting, and APT campaign detection. In light of the observation of artifact leakage, distributional shift, benign syntheticity, and attack covertness in the synthetic datasets [11], we emphasize that the SAGA-generated datasets are not limited to single-campaign training or evaluation. They can be flexibly composed to include multiple campaigns, time spans, and benign activities, allowing researchers to design training and testing splits that better reflect real-world conditions and capture diverse, multi-stage attack behaviors. Furthermore, SAGA enables researchers to explore distributional shifts by generating stage-specific audit logs, supporting the training and evaluation of models on early-stage versus late-stage attack behaviors.

The authenticity of SAGA-generated logs is grounded in two factors: the faithful implementation of attack techniques and the construction of realistic attack sequences. SAGA relies on a red-team emulator (i.e., Caldera) to execute concrete implementations of ATT&CK techniques. The more faithfully the emulator reproduces adversary behaviors, the more realistic the resulting logs. For sequence construction, SAGA supports both expert-instructed campaigns, emulating real-world APT scenarios as defined by domain experts (similar to manual red-teaming), and sampled sequences, assembled from statistically plausible and logically coherent technique transitions. This design enables

SAGA to generate scalable and scenario-driven audit logs, addressing the broader need for labeled and scenario-driven log data for learning-based APT detection research.

The quality of SAGA datasets depends on access to extensive, high-quality campaign knowledge and diverse Technique implementations. The current implementation of SAGA uses 169 abilities and 80 Techniques, with ongoing efforts to expand these numbers to enhance the usefulness of SAGA datasets and cover more sophisticated APT campaigns. While the ATT&CK framework is continuously evolving, SAGA is designed to be extensible. New techniques can be integrated by either updating Caldera with additional abilities as they become available, or manually constructing new attack templates through expert-guided red-team emulation. Furthermore, we also recognize the potential of advanced tools such as large language models (LLMs) used in [53] to assist in the automatic attack generation – a promising direction for future research. Given the inherent differences between synthetic and real-world data, we caution that experimental results obtained using SAGA should be interpreted carefully, as synthetic logs may not present in real environments. As highlighted in [11], careful evaluation is necessary to avoid overestimating generalization capabilities due to unintended signals in synthetic datasets. In this work, the Windows operating system is used as the target environment; however, the concept can be readily applied to other platforms as well.

REFERENCES

- [1] Y.-T. Huang, C. Y. Lin, Y.-R. Guo, K.-C. Lo, Y. S. Sun, and M. C. Chen, "Open source intelligence for malicious behavior discovery and interpretation," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 2, pp. 776–789, Mar./Apr. 2022.
- [2] G.-W. Wong, Y.-T. Huang, Y.-R. Guo, Y. Sun, and M. C. Chen, "Attention-based API locating for malware techniques," *IEEE Trans. Inf. Forensics Secur.*, vol. 19, pp. 1199–1212, 2024.
- [3] M. Zipperle, F. Gottwalt, E. Chang, and T. Dillon, "Provenance-based intrusion detection systems: A survey," *ACM Comput. Surv.*, vol. 55, pp. 1–36, 2022.
- [4] M. Engenuity, "Att&ck evaluations," 2024. [Online]. Available: <https://attacker.mitre-engenuity.org/>
- [5] A. Alsaheel et al., "{ATLAS}: A sequence-based learning approach for attack investigation," in *Proc. 30th USENIX Secur. Symp.*, 2021, pp. 3005–3022.
- [6] H. Ding, J. Zhai, Y. Nan, and S. Ma, "{AIRTAG}: Towards automated attack investigation by unsupervised learning with log texts," in *Proc. 32nd USENIX Secur. Symp.*, 2023, pp. 373–390.
- [7] Z. Cheng et al., "Kairos: Practical intrusion detection and investigation using whole-system provenance," 2023, *arXiv:2308.05034*.
- [8] DARPA, "Transparent computing engagement," 2021. [Online]. Available: <https://github.com/darpa-i2o/Transparent-Computing>
- [9] DARPA, "Operationally transparent cyber (optc) data release," 2021. [Online]. Available: <https://github.com/FiveDirections/OpTC-data>
- [10] A. D. Kent, "Comprehensive, multi-source cyber-security events data set," Los Alamos National Lab. (LANL), Los Alamos, NM (United States), Tech. Rep. LA-UR-15-23810, 2015.
- [11] J. Liu, M. A. Inam, A. Goyal, A. Riddle, K. Westfall, and A. Bates, "What we talk about when we talk about logs: Understanding the effects of dataset quality on endpoint threat detection research," in *Proc. 2025 IEEE Symp. Secur. Privacy*, 2025, pp. 112–129.
- [12] D. Arp et al., "DoS and don'ts of machine learning in computer security," in *Proc. 31st USENIX Secur. Symp.*, 2022, pp. 3971–3988.
- [13] A. S. Jacobs, R. Beltiukov, W. Willinger, R. A. Ferreira, A. Gupta, and L. Z. Granville, "AI/ml for network security: The emperor has no clothes," in *Proc. 2022 ACM SIGSAC Conf. Comput. Commun. Secur.*, 2022, pp. 1537–1551.
- [14] Lockheed Martin Corporation, "Gaining the advantage: Applying cyber kill chain methodology to network defense," 2015. [Online]. Available: https://www.lockheedmartin.com/content/dam/lockheed-martin/rms/documents/cyber/Gaining_the_Advantage_Cyber_Kill_Chain.pdf
- [15] B. Strom, A. Applebaum, D. Miller, K. Nickels, A. Pennington, and C. Thomas, "MITRE ATT&CK: Design and philosophy," 2018. [Online]. Available: <https://www.mitre.org/sites/default/files/publications/pr-18-0944-11-mitre-attack-design-and-philosophy.pdf>
- [16] M. Google Cloud, "Targeted attack life cycle," Accessed: Aug. 24, 2020. [Online]. Available: <https://cloud.google.com/security/resources/insights/targeted-attack-lifecycle>
- [17] APT28, "Cve-2023-38831 exploited by pro-Russia hacking groups in ru-ua conflict zone for credential harvesting operations," 2023. [Online]. Available: <https://blog.cluster25.duskribe.com/2023/10/12/cve-2023-38831-russian-attack>
- [18] A. Applebaum, D. Miller, B. Strom, C. Korban, and R. Wolf, "Intelligent, automated red team emulation," in *Proc. 32nd Annu. Conf. Comput. Secur. Appl.*, 2016, pp. 363–373.
- [19] "Mitre caldera," 2023, [Online]. Available: <https://github.com/mitre/caldera>
- [20] "Atomic red team," 2023, [Online]. Available: <https://atomicredteam.io/>
- [21] R. Bordet and other contributors, "Faker: Python package that generates fake data," 2024. [Online]. Available: <https://faker.readthedocs.io>
- [22] "Vx underground," 2023. [Online]. Available: <https://vx-underground.org/>
- [23] "Virus total," 2023. [Online]. Available: <https://www.virustotal.com/gui/home/upload>
- [24] S. Dataset, "The synthetic audit log dataset with APT campaign," 2024. [Online]. Available: <https://saga-tw.github.io/dataset/>
- [25] E. Manzoor, S. M. Milajerdi, and L. Akoglu, "Fast memory-efficient anomaly detection in streaming heterogeneous graphs," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2016, pp. 1035–1044.
- [26] Malwarebytes, "New lnk attack tied to higaisa APT discovered," 2020. [Online]. Available: <https://www.malwarebytes.com/blog/news/2020/06/higaisa>
- [27] Mandiant, "China-based cyber threat group uses dropbox for malware communications and targets Hong Kong media outlets," 2015. [Online]. Available: <https://www.mandiant.com/resources/blog/china-based-threat>
- [28] Mandiant, "Fin7 evolution and the phishing lnk," 2017. [Online]. Available: <https://www.mandiant.com/resources/blog/fin7-phishing-lnk>
- [29] Ptsecurity, "Cobalt strikes back: An evolving multinational threat to finance," 2017. [Online]. Available: <https://www.ptsecurity.com/upload/corporate/ww-en/analytics/Cobalt-2017-eng.pdf>
- [30] CERT-EE, "Gamaredon infection: From Dropper to entry," 2021. [Online]. Available: https://www.riaa.ee/sites/default/files/content-editors/kuberturvtale_of_gamaredon_infection.pdf
- [31] Cymmetria, "Unveiling patchwork-the copy-paste apt," 2016. [Online]. Available: https://web.archive.org/web/20180825085952/https://s3-us-west-2.amazonaws.com/cymmetria-blog/public/Unveiling_Patchwork.pdf
- [32] Unit 42, "The gorgon group: Slithering between nation state and cybercrime," 2018. [Online]. Available: <https://unit42.paloaltonetworks.com/unit42-gorgon-group-slithering-nation-state-cybercrime/>
- [33] E. F. T. K. Sang and F. D. Meulder, "Introduction to the conll-2003 shared task: Language-independent named entity recognition," 2003, *arXiv:1909.01315*.
- [34] X. Han, T. Pasquier, A. Bates, J. Mickens, and M. Seltzer, "Unicorn: Runtime provenance-based detector for advanced persistent threats," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2020.
- [35] "Sigma rule repository," 2024. [Online]. Available: <https://github.com/SigmaHQ/sigma>
- [36] Y.-T. Huang, Y.-R. Guo, G.-W. Wong, and M. C. Chen, "A cascade approach for apt campaign attribution in system event logs: Technique hunting and subgraph matching," 2024. [Online]. Available: <https://arxiv.org/abs/2410.22602>
- [37] A. Virkud, M. A. Inam, A. Riddle, J. Liu, G. Wang, and A. Bates, "How does endpoint detection use the {mitre}{att&ck} framework," in *Proc. 33rd USENIX Secur. Symp.*, 2024, pp. 3891–3908.
- [38] A. D. Keromytis, "Transparent computing engagement 3 data release," 2021. [Online]. Available: <https://github.com/darpa-i2o/Transparent-Computing/blob/master/README-E3.md>
- [39] J. Glasser and B. Lindauer, "Bridging the gap: A pragmatic approach to generating insider threat data," in *Proc. 2013 IEEE Secur. Privacy Workshops*, 2013, pp. 98–104.

- [40] G. Creech, “Developing a high-accuracy cross platform host-based intrusion detection system capable of reliably detecting zero-day attacks,” Ph.D. dissertation, UNSW Sydney, 2014.
- [41] S. Myneni et al., “Dapt 2020-constructing a benchmark dataset for advanced persistent threats,” in *Proc. Deployable Mach. Learn. Secur. Defense: First Int. Workshop*, San Diego, CA, USA, 2020, pp. 13–163.
- [42] A. Berady, M. Jaume, V. V. T. Tong, and G. Guette, “PWNJUTSU: A dataset and a semantics-driven approach to retrace attack campaigns,” *IEEE Trans. Netw. Service Manag.*, vol. 19, no. 4, pp. 5252–5264, Dec. 2022.
- [43] S. Myneni et al., “Unraveled—a semi-synthetic dataset for advanced persistent threats,” *Comput. Netw.*, vol. 227, 2023, Art. no. 109688.
- [44] J. Gjerstad, F. Kadiric, G. Grov, E. Kjellstadli, and M. L. Asprusten, “LADEMU: A modular & continuous approach for generating labelled apt datasets from emulations,” in *Proc. 2022 IEEE Int. Conf. Big Data*, 2022, pp. 2610–2619.
- [45] J. Zengy et al., “SHADEWATCHER: Recommendation-guided cyber threat analysis using system audit records,” in *Proc. 2022 IEEE Symp. Secur. Privacy*, 2022, pp. 489–506.
- [46] Z. Jia, Y. Xiong, Y. Nan, Y. Zhang, J. Zhao, and M. Wen, “Magic: Detecting advanced persistent threats via masked graph representation learning,” 2023, *arXiv:2310.09831*.
- [47] M. Sharif et al., “DrSec: Flexible distributed representations for efficient endpoint security,” in *Proc. 2024 IEEE Symp. Secur. Privacy*, 2024, pp. 145–145.
- [48] J. Zeng, Z. L. Chua, Y. Chen, K. Ji, Z. Liang, and J. Mao, “WATSON: Abstracting behaviors from audit logs via aggregation of contextual semantics,” in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2021.
- [49] S. M. Milajerdi, R. Gjosec, B. Eshete, R. Sekar, and V. Venkatakrishnan, “HOLMES: Real-time APT detection through correlation of suspicious information flows,” in *Proc. IEEE Symp. Secur. Privacy*, 2019, pp. 1137–1152.
- [50] W. U. Hassan, A. Bates, and D. Marino, “Tactical provenance analysis for endpoint detection and response systems,” in *Proc. IEEE Symp. Secur. Privacy*, 2020, pp. 1172–1189.
- [51] K. Kurniawan, A. Ekelhart, E. Kiesling, G. Quirchmayr, and A. M. Tjoa, “Krystal: Knowledge graph-based framework for tactical attack discovery in audit data,” *Comput. Secur.*, vol. 121, 2022, Art. no. 102828.
- [52] “Sigma - generic signature format for siem systems,” 2024. [Online]. Available: <https://github.com/SigmaHQ/sigma>
- [53] B. Singer, K. Lucas, L. Adiga, M. Jain, L. Bauer, and V. Sekar, “On the feasibility of using LLMS to autonomously execute multi-host network attacks,” 2025, *arXiv:2501.16466*.



Yi-Ting Huang received the PhD degree in information management from National Taiwan University, in 2015. She is an assistant professor with the National Taiwan University of Science and Technology. She was a postdoctoral fellow at Academia Sinica. Her primary research interests include malware analysis, cyber threat intelligence analysis, deep learning, and natural language processing.



Ying-Ren Guo is a research assistant in Academia Sinica. His research interests include malware reverse engineering and analysis, and cybersecurity.



Yu-Sheng Yang is currently working toward the master's degree in computer science with National Taiwan University. His research focuses on large language models (LLMs) and computer vision.



Guo-Wei Wong received the master's degree in thermal and fluid science from the National Yunlin University of Science and Technology, in 2017. He is currently working towards the PhD degree of CSIE with the National Taiwan University. His primary research interests include deep learning in fluid dynamics and for complicated applications, including malware analysis and PM2.5 prediction.



Yu-Zih Jheng is currently working toward the master's degree in Information management with National Taiwan University. Her research interests include static and dynamic malware analysis, red teaming, and penetration testing.



services, and performance modeling and evaluation.

Yeali S. Sun received the BS degree from the Computer Science and Information Engineering department of National Taiwan University, and the MS and PhD degrees in computer science from the University of California, Los Angeles (UCLA). From 1988 to 1993, she was with Bell Communications Research Inc. In August 1993, she joined National Taiwan University and is currently a professor of the Department of Information Management. Her research interests are in the areas of Internet security and forensics, Quality of Service (QoS), cloud computing and services, and performance modeling and evaluation.



approaches to resilience.

Jessemyn Modini is an IT leader and cyber security researcher. Jess holds 5 master's qualifications in Cybersecurity Operations, Systems Engineering, Space Operations, Project Management and International Relations (Security). Jess is currently undertaking a doctorate in Cyber Security, focusing on applications of epidemiology to cyber security (Cyber Epidemiology) for threat modelling and cyber herd immunity at scale. Jess is currently a senior Technologist with Amazon Web Services where she drives global technology strategy. Prior to AWS, Jess was the Australian Cyber Security Centre (ACSC) Engineering Manager. Jess is passionate about security, global cyber threat intelligence sharing and multidisciplinary approaches to resilience.



Timothy Lynar received the PhD degree from the University of Newcastle, Australia, in 2011. He is a senior lecturer with UNSW Canberra. He worked at IBM research for 7.5 years where he was a research staff member and master inventor, before joining UNSW, in 2019. Tim's primary research focus is the application of machine learning to Cyber security. Tim has a background in simulation, modelling and distributed computing including cloud and IoT systems.



Meng Chang Chen received the PhD degree in computer science from the University of California, Los Angeles. He was with AT&T Bell Labs and is now a research fellow/professor of research center for information technology innovation, Academia Sinica, Taiwan. His current research interests include computer and network security, wireless network, deep learning for complicated applications, data and knowledge engineering.